

# **LAPORAN PRATIKUM STRUKTUR DATA**

**8**



Dosen Pengampu:

DR. Wahyudi. .S.T.M.T

Asisten Lab:

Jovantri Immanuel Gulo

Disusun Oleh:

Marcello Bayu Denar Widjaksono

2511532011

Fakultas Teknologi Informasi

Departemen Informatika

Universitas Andalas

2026

## **KATA PENGANTAR**

Puji dan syukur senantiasa penulis panjatkan kepada Allah SWT yang telah melimpahkan rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan ini guna memenuhi laporan pratikum mata kuliah Struktur Data.

Pada kesempatan ini, Penulis mengucapkan terima kasih Bapak DR. Wahyudi. .S.T.M.T yang telah memberikan tugas laporan pratikum. Penulis juga ingin mengucapkan terima kasih kepada pihak-pihak yang telah mendukung serta membantu penulis dalam penyelesaian laporan pratikum Struktur Data.

Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari sempurna dikarenakan terbatasnya pengalaman dan pengetahuan yang penulis miliki. Oleh karena itu, penulis mengharapkan segala bentuk saran dan masukan serta kritik yang membangun dari berbagai pihak.

Akhir kata, penulis berharap Laporan ini dapat memberikan manfaat bagi perkembangan dunia pendidikan.



# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Dalam ilmu komputer dan struktur data, pengurutan (sorting) merupakan salah satu operasi fundamental yang paling sering digunakan untuk mengorganisir data agar lebih mudah dicari, dianalisis, atau diproses lebih lanjut. Terdapat berbagai macam algoritma pengurutan dengan karakteristik kompleksitas waktu dan ruang yang berbeda-beda, seperti Insertion Sort, Selection Sort, dan Bubble Sort. Pemahaman mendalam mengenai logika kerja masing-masing algoritma sangat penting bagi mahasiswa informatika untuk dapat memilih metode yang paling efisien sesuai dengan konteks permasalahan.

Selain implementasi algoritma berbasis konsol, visualisasi proses pengurutan melalui Graphical User Interface (GUI) menjadi metode yang efektif untuk memahami bagaimana data berpindah dan dibandingkan secara step-by-step. Melalui praktikum ini, mahasiswa tidak hanya dituntut untuk menulis kode pengurutan yang fungsional, tetapi juga membangun aplikasi interaktif menggunakan Java Swing untuk memvisualisasikan algoritma Insertion Sort. Hal ini bertujuan untuk menjembatani pemahaman teoritis dengan implementasi nyata serta meningkatkan kemampuan pemrograman visual.

### **1.2 Tujuan Pratikum**

1. Mengimplementasikan algoritma pengurutan merge Sort, Shell Sort, dan quick Sort menggunakan bahasa pemrograman Java.
2. Memahami logika perbandingan dan pertukaran elemen pada masing-masing algoritma pengurutan.
3. Membangun aplikasi GUI menggunakan Java Swing untuk memvisualisasikan proses quick Sort dan merge sort secara langkah demi langkah.
4. Menganalisis perubahan array sebelum dan setelah proses pengurutan dilakukan.

### **1.3 Manfaat Pratikum**

1. Mahasiswa mampu mengimplementasikan algoritma sorting dasar secara mandiri dalam bentuk kode program.
2. Mahasiswa memahami mekanisme in-place sorting dimana pengurutan dilakukan tanpa memerlukan array tambahan yang signifikan.
3. Mahasiswa mendapatkan pengalaman praktis dalam mengembangkan antarmuka grafis (GUI) menggunakan komponen Swing seperti JFrame, JPanel, JButton, dan JLabel.
4. Visualisasi GUI membantu mahasiswa untuk melihat secara nyata bagaimana elemen digeser dan disisipkan pada algoritma quick sort, merge sort, sehingga memperkuat pemahaman konseptual.

## BAB II PEMBAHASAN

### 2.1 Pratikum Striktur Data

```
public class ShellSort_2511532011 {

    public static void ShellSort_2011(int[] A_2011) {
        int n_2011 = A_2011.length;
        int gap_2011 = n_2011/2;
        while (gap_2011 > 0) {
            for (int i_2011 = gap_2011; i_2011 < n_2011; i_2011++) {
                int temp_2011 = A_2011[i_2011];
                int j_2011 = i_2011;
                while (j_2011 >= gap_2011 && A_2011[j_2011 - gap_2011] > temp_2011) {
                    A_2011[j_2011] = A_2011[j_2011 - gap_2011];
                    j_2011 = j_2011 - gap_2011;
                }
                A_2011[j_2011] = temp_2011;
            }
            gap_2011 = gap_2011 / 2;
        }
    }

    public static void main(String[] args) {
        int[] data_2011 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};

        System.out.print("Sebelum : ");
        printArray_2011(data_2011);

        ShellSort_2011(data_2011);

        System.out.print("Sesudah (Shell Sort) : ");
        printArray_2011(data_2011);
    }

    public static void printArray_2011(int[] arr_2011) {
        for (int i_2011 : arr_2011)
            System.out.print(i_2011 + " ");
        System.out.println();
    }
}
```

## Pembahasan Class ShellSort:

### 1. Deklarasi Kelas

`public class ShellSort_2511532011`: Kelas publik yang berfungsi sebagai wadah implementasi algoritma Shell Sort dan driver pengujian.

Semua metode dideklarasikan static karena kelas ini berfungsi sebagai utility class tanpa perlu instansiasi objek.

### 2. Metode ShellSort\_2011 (Inti Algoritma)

`public static void ShellSort_2011(int[] A_2011)`: Metode statis yang menerima array integer dan melakukan pengurutan secara in-place menggunakan paradigma Shell Sort.

`int n_2011 = A_2011.length`:: Menyimpan panjang array ke variabel lokal untuk efisiensi akses di dalam loop.

`int gap_2011 = n_2011/2`:: Inisialisasi Gap. Shell Sort bekerja dengan membandingkan elemen yang berjarak gap. Dimulai dari setengah panjang array, kemudian gap diperkecil secara bertahap.

### 3. Outer Loop: Pengurangan Gap

`while (gap_2011 > 0)`: Loop utama yang terus berjalan selama nilai gap masih positif.

Setiap iterasi outer loop mewakili satu "pass" dengan ukuran gap tertentu.

`gap_2011 = gap_2011 / 2`:: Gap Reduction. Di akhir setiap pass, gap dibagi 2 (integer division). Sequence gap yang digunakan:  $n/2$ ,  $n/4$ ,  $n/8$ , ..., 1.

Ketika `gap_2011 = 1`, algoritma berubah menjadi Insertion Sort standar, namun pada data yang sudah "hampir terurut" akibat pass sebelumnya.

### 4. Middle Loop: Traversal Sub-arrays

`for (int i_2011 = gap_2011; i_2011 < n_2011; i_2011++)`: Loop yang memproses setiap elemen mulai dari index gap hingga akhir array.

Konsep Sub-array: Dengan gap tertentu, array dapat dipandang sebagai beberapa sub-array independen. Contoh: jika `gap=3`, maka elemen index `[0,3,6,9]` membentuk satu sub-array, `[1,4,7,10]` sub-array lain, dst.

Loop ini memastikan setiap elemen menjadi "key" yang akan disisipkan ke posisi yang benar dalam sub-array-nya.

5. Inner Loop: Insertion Sort dengan Gap

`int temp_2011 = A_2011[i_2011];` Simpan elemen kunci (key) yang akan disisipkan ke variabel temporary.

`int j_2011 = i_2011;` Inisialisasi pointer j pada posisi elemen kunci.

`while (j_2011 >= gap_2011 && A_2011[j_2011 - gap_2011] > temp_2011);` Logika Shift dengan Gap.

`j_2011 >= gap_2011;` Pastikan pointer tidak keluar dari batas kiri sub-array.

`A_2011[j_2011 - gap_2011] > temp_2011;` Bandingkan elemen di posisi j-gap (elemen sebelumnya dalam sub-array) dengan key.

Jika elemen sebelumnya lebih besar, maka perlu digeser.

`A_2011[j_2011] = A_2011[j_2011 - gap_2011];` Geser elemen yang lebih besar sejauh gap posisi ke kanan.

`j_2011 = j_2011 - gap_2011;` Gerakkan pointer j mundur sejauh gap untuk melanjutkan perbandingan dalam sub-array.

`A_2011[j_2011] = temp_2011;` Insertion Step. Setelah menemukan posisi yang tepat, sisipkan key ke posisi j.

6. Metode `printArray_2011` (Helper Cetak Array)

`public static void printArray_2011(int[] arr_2011);` Metode utility untuk menampilkan isi array ke konsol.

`for (int i_2011 : arr_2011);` Menggunakan enhanced for-loop (for-each) untuk iterasi yang lebih ringkas.

`System.out.print(i_2011 + " ");` Cetak setiap elemen diikuti spasi.

`System.out.println();` Pindah baris setelah semua elemen tercetak.

7. Metode main - Inisialisasi Data Uji

`int[] data_2011 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};` Mendeklarasikan array dengan 10 elemen acak sebagai data uji.

Array ini sengaja dibuat tidak teratur untuk mendemonstrasikan efektivitas algoritma.

8. Metode main - Eksekusi dan Output

`System.out.print("Sebelum : "); printArray_2011(data_2011);` Tampilkan kondisi awal array sebelum sorting.



ShellSort\_2011(data\_2011);: Panggil metode Shell Sort untuk mengurutkan array secara in-place.

System.out.print("Sesudah (Shell Sort) : "); printArray\_2011(data\_2011);:

Tampilkan array setelah pengurutan selesai.

```

public class QuickSort_2511532011 {

    static void swap_2011(int[] arr_2011, int i_2011, int j_2011) {
        int temp_2011 = arr_2011[i_2011];
        arr_2011[i_2011] = arr_2011[j_2011];
        arr_2011[j_2011] = temp_2011;
    }

    static void medianOfThree(int[] arr_2011, int low_2011, int high_2011) {
        int mid_2011 = low_2011 + (high_2011 - low_2011) / 2;

        if (arr_2011[low_2011] > arr_2011[mid_2011]) {
            swap_2011(arr_2011, low_2011, mid_2011);
        }
        if (arr_2011[low_2011] > arr_2011[high_2011]) {
            swap_2011(arr_2011, low_2011, high_2011);
        }
        if (arr_2011[mid_2011] > arr_2011[high_2011]) {
            swap_2011(arr_2011, mid_2011, high_2011);
        }
        swap_2011(arr_2011, mid_2011, high_2011);
    }

    static int partition_2011(int[] arr_2011, int low_2011, int high_2011) {

        medianOfThree(arr_2011, low_2011, high_2011);

        int pivot_2011 = arr_2011[high_2011];
        int i_2011 = (low_2011 - 1);

        for (int j_2011 = low_2011; j_2011 <= high_2011 - 1; j_2011++) {

            if (arr_2011[j_2011] < pivot_2011) {

                i_2011++;
                swap_2011(arr_2011, i_2011, j_2011);
            }
        }
        swap_2011(arr_2011, i_2011 + 1, high_2011);
        return (i_2011 + 1);
    }

    static void quickSort_2011(int[] arr_2011, int low_2011, int high_2011) {
        if (low_2011 < high_2011) {

```

```

        int pi_2011 = partition_2011(arr_2011, low_2011, high_2011);
        quickSort_2011(arr_2011, low_2011, pi_2011 - 1);
        quickSort_2011(arr_2011, pi_2011 + 1, high_2011);
    }
}

public static void printArr(int[] arr_2011) {
    for (int i_2011 = 0; i_2011 < arr_2011.length; i_2011++) {
        System.out.print(arr_2011[i_2011] + " ");
    }
    System.out.println();
}

public static void main(String[] args) {
    int[] arr_2011 = {10, 7, 8, 9, 1, 5};
    int N_2011 = arr_2011.length;
    System.out.print("Data sebelum diurutkan : ");
    printArr(arr_2011);

    quickSort_2011(arr_2011, 0, N_2011 - 1);

    System.out.print("Data Terurut quick sort: ");
    printArr(arr_2011);
}
}

```

## Pembahasan Class QuickSort:

### 1. Deklarasi Kelas

public class QuickSort\_2511532011: Kelas publik yang berfungsi sebagai wadah implementasi algoritma Quick Sort dan driver pengujian.

Semua metode dideklarasikan static karena kelas ini berfungsi sebagai utility class tanpa perlu instansiasi objek.

### 2. Metode swap\_2011 (Helper Tukar Elemen)

static void swap\_2011(int[] arr\_2011, int i\_2011, int j\_2011): Metode helper untuk menukar dua elemen dalam array pada index i dan j.

int temp\_2011 = arr\_2011[i\_2011];: Simpan nilai elemen pertama ke variabel temporary.

arr\_2011[i\_2011] = arr\_2011[j\_2011];: Timpa posisi i dengan nilai dari posisi j.

`arr_2011[j_2011] = temp_2011;` Isi posisi `j` dengan nilai yang disimpan di `temporary`.

Metode ini digunakan berulang kali di berbagai bagian algoritma untuk operasi pertukaran elemen.

3. Metode `medianOfThree` (Optimasi Pemilihan Pivot)

`static void medianOfThree(int[] arr_2011, int low_2011, int high_2011):`  
Metode tambahan untuk memilih pivot yang lebih baik menggunakan strategi Median-of-Three.

`int mid_2011 = low_2011 + (high_2011 - low_2011) / 2;` Hitung index tengah sub-array. Rumus ini menghindari potensi integer overflow dibanding `(low + high) / 2`.

4. Tiga Perbandingan untuk Mengurutkan low-mid-high:

`if (arr_2011[low_2011] > arr_2011[mid_2011]):` Jika elemen `low` > `mid`, tukar keduanya.

`if (arr_2011[low_2011] > arr_2011[high_2011]):` Jika elemen `low` > `high`, tukar keduanya.

`if (arr_2011[mid_2011] > arr_2011[high_2011]):` Jika elemen `mid` > `high`, tukar keduanya.

Setelah tiga perbandingan ini, ketiga elemen (`low`, `mid`, `high`) terurut relatif: `arr[low] ≤ arr[mid] ≤ arr[high]`.

`swap_2011(arr_2011, mid_2011, high_2011);` Penting: Pindahkan elemen median (yang sekarang berada di `mid`) ke posisi `high`. Ini mempersiapkan pivot untuk algoritma `partition` Lomuto yang menggunakan elemen terakhir sebagai pivot.

Manfaat: Mencegah worst-case  $O(n^2)$  pada data yang sudah terurut atau hampir terurut, karena pivot tidak lagi selalu elemen terkecil/terbesar.

5. Metode `partition_2011` (Inti Pembagian Partisi)

`static int partition_2011(int[] arr_2011, int low_2011, int high_2011):`  
Metode yang membagi array menjadi dua partisi berdasarkan nilai pivot.

`medianOfThree(arr_2011, low_2011, high_2011);` Panggil optimasi pivot sebelum memulai partisi.

int pivot\_2011 = arr\_2011[high\_2011];: Ambil nilai pivot dari posisi high (setelah median-of-three memindahkannya ke sana).

int i\_2011 = (low\_2011 - 1);: Inisialisasi pointer i di sebelah kiri batas sub-array. Pointer ini menandai posisi terakhir elemen yang lebih kecil dari pivot.

for (int j\_2011 = low\_2011; j\_2011 <= high\_2011 - 1; j\_2011++): Loop traversing melalui semua elemen kecuali pivot (yang berada di high).

if (arr\_2011[j\_2011] < pivot\_2011): Jika elemen saat ini lebih kecil dari pivot:

i\_2011++;: Geser pointer i ke kanan (menyiapkan posisi untuk elemen kecil).

swap\_2011(arr\_2011, i\_2011, j\_2011);: Tukar elemen kecil ke posisi i, sehingga semua elemen di kiri i terjamin < pivot.

swap\_2011(arr\_2011, i\_2011 + 1, high\_2011);: Final Placement. Setelah loop selesai, tempatkan pivot di posisi yang benar (i+1), di mana semua elemen di kirinya < pivot dan semua elemen di kanannya  $\geq$  pivot.

return (i\_2011 + 1);: Kembalikan index posisi final pivot untuk digunakan dalam rekursi.

#### 6. Metode quickSort\_2011 (Rekursi Divide-and-Conquer)

static void quickSort\_2011(int[] arr\_2011, int low\_2011, int high\_2011): Metode rekursif utama yang menerapkan paradigma Divide-and-Conquer.

if (low\_2011 < high\_2011): Base Case. Rekursi berlanjut hanya jika sub-array memiliki minimal 2 elemen. Jika low  $\geq$  high, sub-array sudah terurut (0 atau 1 elemen).

int pi\_2011 = partition\_2011(arr\_2011, low\_2011, high\_2011);: Divide: Partisi array dan dapatkan index pivot yang sudah di posisi final.

quickSort\_2011(arr\_2011, low\_2011, pi\_2011 - 1);: Conquer Kiri: Rekursif urutkan sub-array di sebelah kiri pivot.

quickSort\_2011(arr\_2011, pi\_2011 + 1, high\_2011);: Conquer Kanan: Rekursif urutkan sub-array di sebelah kanan pivot.

Catatan: Pivot tidak perlu diproses lagi karena sudah berada di posisi akhir yang benar.

7. Metode printArr (Helper Cetak Array)

`public static void printArr(int[] arr_2011):` Metode utility untuk menampilkan isi array ke konsol.

`for (int i_2011 = 0; i_2011 < arr_2011.length; i_2011++):` Loop melalui semua elemen array.

`System.out.print(arr_2011[i_2011] + " ");` Cetak setiap elemen diikuti spasi.

`System.out.println();` Pindah baris setelah semua elemen tercetak.

8. Metode main - Inisialisasi Data Uji

`int[] arr_2011 = {10, 7, 8, 9, 1, 5};` Mendeklarasikan array dengan 6 elemen acak sebagai data uji.

`int N_2011 = arr_2011.length;` Simpan panjang array untuk parameter pemanggilan metode sorting.

9. Metode main - Eksekusi dan Output

`System.out.print("Data sebelum diurutkan : "); printArr(arr_2011);` Tampilkan kondisi awal array.

`quickSort_2011(arr_2011, 0, N_2011 - 1);` Panggil metode Quick Sort dengan batas awal `low=0` dan `high=5` (seluruh array).

`System.out.print("Data Terurut quick sort: "); printArr(arr_2011);` Tampilkan array setelah pengurutan selesai.

```

public class MergeSort_2511532011 {

    void merge_2011(int arr_2011[], int l_2011, int m_2011, int r_2011) {

        int n1_2011 = m_2011 - l_2011 + 1;
        int n2_2011 = r_2011 - m_2011;

        int L_2011[] = new int[n1_2011];
        int R_2011[] = new int[n2_2011];

        for (int i_2011 = 0; i_2011 < n1_2011; ++i_2011) {
            L_2011[i_2011] = arr_2011[l_2011 + i_2011];
        }

        for (int j_2011 = 0; j_2011 < n2_2011; ++j_2011) {
            R_2011[j_2011] = arr_2011[m_2011 + 1 + j_2011];
        }

        int i_2011 = 0, j_2011 = 0;

        int k_2011 = l_2011;
        while (i_2011 < n1_2011 && j_2011 < n2_2011) {
            if (L_2011[i_2011] <= R_2011[j_2011]) {
                arr_2011[k_2011] = L_2011[i_2011];
                i_2011++;
            } else {
                arr_2011[k_2011] = R_2011[j_2011];
                j_2011++;
            }
            k_2011++;
        }

        while (i_2011 < n1_2011) {
            arr_2011[k_2011] = L_2011[i_2011];
            i_2011++;
            k_2011++;
        }

        while (j_2011 < n2_2011) {
            arr_2011[k_2011] = R_2011[j_2011];
            j_2011++;
            k_2011++;
        }
    }
}

```

```

void sort_2011(int arr[], int l_2011, int r_2011) {
    if (l_2011 < r_2011) {

        int m_2011 = (l_2011 + r_2011) / 2;

        sort_2011(arr, l_2011, m_2011);
        sort_2011(arr, m_2011 + 1, r_2011); /

        merge_2011(arr, l_2011, m_2011, r_2011);
    }
}

static void printArray_2011(int arr_2011[]) {
    int n_2011 = arr_2011.length;
    for (int i_2011 = 0; i_2011 < n_2011; i_2011++) {
        System.out.print(arr_2011[i_2011] + " ");
    }
    System.out.println();
}

public static void main(String[] args) {
    int arr_2011[] = {12, 11, 13, 5, 6, 7};
    System.out.println("Sebelum terurut : ");
    printArray_2011(arr_2011);

    MergeSort_2511532011 ob_2011 = new MergeSort_2511532011();

    ob_2011.sort_2011(arr_2011, 0, arr_2011.length - 1);
    System.out.println("\nSesudah Terurut menggunakan Merge SOrt :");
    printArray_2011(arr_2011);
}
}

```

## Pembahasan Class MergeSort:

### 1. Deklarasi Kelas

public class MergeSort\_2511532011: Kelas publik yang berfungsi sebagai wadah implementasi algoritma Merge Sort.

Kelas ini tidak menggunakan metode static untuk merge\_2011 dan sort\_2011, sehingga memerlukan instansiasi objek untuk pemanggilan metode (berbeda dengan implementasi sorting sebelumnya).

### 2. Metode merge\_2011 (Inti Penggabungan Dua Sub-array)



`void merge_2011(int arr_2011[], int l_2011, int m_2011, int r_2011):`  
Metode yang menggabungkan dua sub-array yang sudah terurut menjadi satu array terurut.

Parameter:

`arr_2011[]`: Array utama yang akan dimodifikasi.

`l_2011`: Index kiri (left) batas sub-array pertama.

`m_2011`: Index tengah (middle) pemisah dua sub-array.

`r_2011`: Index kanan (right) batas sub-array kedua.

3. Inisialisasi Ukuran Sub-array

`int n1_2011 = m_2011 - l_2011 + 1;` Hitung jumlah elemen sub-array kiri (dari `l` sampai `m`, inklusif).

`int n2_2011 = r_2011 - m_2011;` Hitung jumlah elemen sub-array kanan (dari `m+1` sampai `r`).

Alokasi Array Temporary

`int L_2011[] = new int[n1_2011];` Buat array temporary kiri untuk menyalin data sub-array pertama.

`int R_2011[] = new int[n2_2011];` Buat array temporary kanan untuk menyalin data sub-array kedua.

4. Copy Data ke Array Temporary

`for (int i_2011 = 0; i_2011 < n1_2011; ++i_2011) { L_2011[i_2011] = arr_2011[l_2011 + i_2011]; }` Salin elemen dari array utama ke array temporary kiri.

`for (int j_2011 = 0; j_2011 < n2_2011; ++j_2011) { R_2011[j_2011] = arr_2011[m_2011 + 1 + j_2011]; }` Salin elemen ke array temporary kanan, dimulai dari index `m+1`.

5. Inisialisasi Pointer Merge

`int i_2011 = 0, j_2011 = 0;` Pointer untuk traversing array temporary `L` dan `R`, dimulai dari index 0.

`int k_2011 = l_2011;` Penting: Pointer untuk menulis hasil merge kembali ke array utama, dimulai dari index `l` (bukan 0), karena merge hanya terjadi pada rentang `[l, r]`.

6. Logika Merge Utama (Comparative Merge)

while (i\_2011 < n1\_2011 && j\_2011 < n2\_2011): Loop berjalan selama kedua array temporary masih memiliki elemen yang belum diproses.

if (L\_2011[i\_2011] <= R\_2011[j\_2011]): Bandingkan elemen saat ini dari kedua array.

Jika elemen kiri  $\leq$  elemen kanan: Ambil elemen kiri.

arr\_2011[k\_2011] = L\_2011[i\_2011];: Tulis elemen kiri ke array utama.

i\_2011++;: Geser pointer array kiri.

else: Jika elemen kanan lebih kecil.

arr\_2011[k\_2011] = R\_2011[j\_2011];: Tulis elemen kanan ke array utama.

j\_2011++;: Geser pointer array kanan.

k\_2011++;: Selalu geser pointer tulis ke array utama setelah setiap penulisan.

Stability: Penggunaan <= (bukan <) menjamin algoritma bersifat stable (elemen sama tetap mempertahankan urutan relatif).

#### 7. Copy Sisa Elemen (Remaining Elements)

while (i\_2011 < n1\_2011) { arr\_2011[k\_2011] = L\_2011[i\_2011];  
i\_2011++; k\_2011++; }: Jika array kiri masih memiliki sisa elemen (artinya semua elemen kanan sudah habis), salin seluruh sisa ke array utama.

while (j\_2011 < n2\_2011) { arr\_2011[k\_2011] = R\_2011[j\_2011];  
j\_2011++; k\_2011++; }: Jika array kanan masih memiliki sisa, salin seluruh sisa ke array utama.

#### 8. Metode sort\_2011 (Rekursi Divide-and-Conquer)

void sort\_2011(int arr[], int l\_2011, int r\_2011): Metode rekursif utama yang membagi array menjadi bagian-bagian lebih kecil.

if (l\_2011 < r\_2011): Base Case. Rekursi berlanjut hanya jika sub-array memiliki minimal 2 elemen. Jika  $l \geq r$ , sub-array sudah terurut (0 atau 1 elemen).

int m\_2011 = (l\_2011 + r\_2011) / 2;: Hitung index tengah untuk membagi array menjadi dua bagian sama besar.

sort\_2011(arr, l\_2011, m\_2011);: Divide Kiri: Rekursif urutkan sub-array kiri dari l sampai m.

sort\_2011(arr, m\_2011 + 1, r\_2011);: Divide Kanan: Rekursif urutkan sub-array kanan dari m+1 sampai r.

merge\_2011(arr, l\_2011, m\_2011, r\_2011);: Conquer: Setelah kedua sub-array terurut, gabungkan keduanya menggunakan metode merge\_2011.

9. Metode printArray\_2011 (Helper Cetak Array)

static void printArray\_2011(int arr\_2011[]): Metode utility statis untuk menampilkan isi array ke konsol.

int n\_2011 = arr\_2011.length;: Simpan panjang array.

for (int i\_2011 = 0; i\_2011 < n\_2011; i\_2011++): Loop melalui semua elemen.

System.out.print(arr\_2011[i\_2011] + " ");: Cetak setiap elemen diikuti spasi.

System.out.println();: Pindah baris setelah selesai.

Metode main - Inisialisasi Data Uji

int arr\_2011[] = {12, 11, 13, 5, 6, 7};: Mendeklarasikan array dengan 6 elemen acak sebagai data uji.

System.out.println("Sebelum terurut : "); printArray\_2011(arr\_2011);: Tampilkan kondisi awal array.

10. Metode main - Instansiasi dan Eksekusi

MergeSort\_2511532011 ob\_2011 = new MergeSort\_2511532011();:

Penting: Karena metode sort\_2011 dan merge\_2011 tidak static, harus dibuat objek instance terlebih dahulu.

ob\_2011.sort\_2011(arr\_2011, 0, arr\_2011.length - 1);: Panggil metode sorting dengan batas low=0 dan high=5 (seluruh array).

System.out.println("\nSesudah Terurut menggunakan Merge Sort :"); printArray\_2011(arr\_2011);: Tampilkan hasil array yang telah terurut.

```
public class BubbleSortGUI_2511532011 extends JFrame {

    private JTextField inputField_2011;

    private JButton setButton_2011;

    private JButton stepButton_2011;

    private JButton resetButton_2011;

    private JTextArea stepArea_2011;

    private JPanel panelArray_2011;

    private int[] array_2011;

    private JLabel[] labelArray_2011;

    private int i_2011 = 0;

    private int j_2011 = 0;

    private int stepCount_2011 = 1;

    private boolean sorting_2011 = false;

    public BubbleSortGUI_2511532011() {

        setTitle("Visualisasi Bubble Sort");

        setSize(800, 500);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        setLocationRelativeTo(null);

        setLayout(new BorderLayout(10, 10));

        // Panel input bagian atas

        JPanel inputPanel = new JPanel(new FlowLayout());

        JLabel inputLabel = new JLabel("Masukkan angka (pisahkan dengan koma):");

        inputField_2011 = new JTextField(30);

        setButton_2011 = new JButton("Set Array");

        stepButton_2011 = new JButton("Step");

        resetButton_2011 = new JButton("Reset");

        stepButton_2011.setEnabled(false);

        inputPanel.add(inputLabel);

        inputPanel.add(inputField_2011);

        inputPanel.add(setButton_2011);

        inputPanel.add(stepButton_2011);

        inputPanel.add(resetButton_2011);

        add(inputPanel, BorderLayout.NORTH);

        // Panel untuk menampilkan array

        panelArray_2011 = new JPanel(new FlowLayout());

        panelArray_2011.setPreferredSize(new Dimension(750, 120));
```

```

add(panelArray_2011, BorderLayout.CENTER);

// Text area untuk menampilkan langkah sorting
stepArea_2011 = new JTextArea();
stepArea_2011.setEditable(false);
stepArea_2011.setFont(new Font("Monospaced", Font.PLAIN, 14));

JScrollPane scrollPane = new JScrollPane(stepArea_2011);
scrollPane.setPreferredSize(new Dimension(750, 250));

add(scrollPane, BorderLayout.SOUTH);

// Event tombol
setButton_2011.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        setArrayFromInput();
    }
});

stepButton_2011.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        performStep();
    }
});

resetButton_2011.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        reset();
    }
});
}

private void setArrayFromInput() {
    String text = inputField_2011.getText().trim();

    if (text.isEmpty()) {
        return;
    }

    String[] parts_2011 = text.split(",");
    array_2011 = new int[parts_2011.length];

    try {
        for (int k = 0; k < parts_2011.length; k++) {

```

```

        array_2011[k] = Integer.parseInt(parts_2011[k].trim());
    }
} catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(
        this,
        "Masukkan hanya angka yang dipisahkan koma!",
        "Error",
        JOptionPane.ERROR_MESSAGE
    );
    return;
}

i_2011 = 0;
j_2011 = 0;
stepCount_2011 = 1;
sorting_2011 = true;

stepButton_2011.setEnabled(true);
stepArea_2011.setText("");
panelArray_2011.removeAll();

labelArray_2011 = new JLabel[array_2011.length];

for (int k_2011 = 0; k_2011 < array_2011.length; k_2011++) {
    labelArray_2011[k_2011] = new JLabel(String.valueOf(array_2011[k_2011]));
    labelArray_2011[k_2011].setFont(new Font("Arial", Font.BOLD, 24));
    labelArray_2011[k_2011].setOpaque(true);
    labelArray_2011[k_2011].setBackground(Color.WHITE);
    labelArray_2011[k_2011].setBorder(BorderFactory.createLineBorder(Color.BLACK));
    labelArray_2011[k_2011].setPreferredSize(new Dimension(50, 50));
    labelArray_2011[k_2011].setHorizontalAlignment(SwingConstants.CENTER);

    panelArray_2011.add(labelArray_2011[k_2011]);
}

panelArray_2011.revalidate();
panelArray_2011.repaint();
}

private void performStep() {
    if (!sorting_2011 || i_2011 >= array_2011.length - 1) {
        sorting_2011 = false;
        stepButton_2011.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
        return;
    }
}

```

```

resetHighlights();

StringBuilder stepLog_2011 = new StringBuilder();

labelArray_2011[j_2011].setBackground(Color.CYAN);
labelArray_2011[j_2011 + 1].setBackground(Color.CYAN);

if (array_2011[j_2011] > array_2011[j_2011 + 1]) {
    // Swap
    int temp_2011 = array_2011[j_2011];
    array_2011[j_2011] = array_2011[j_2011 + 1];
    array_2011[j_2011 + 1] = temp_2011;

    labelArray_2011[j_2011].setBackground(Color.RED);
    labelArray_2011[j_2011 + 1].setBackground(Color.RED);

    stepLog_2011.append("Langkah ").append(stepCount_2011).append(": ")
        .append("Menukar elemen ke-").append(j_2011)
        .append(" (").append(array_2011[j_2011 + 1]).append(")")
        .append(" dengan ke-").append(j_2011 + 1)
        .append(" (").append(array_2011[j_2011]).append(")\n");
} else {
    stepLog_2011.append("Langkah ").append(stepCount_2011).append(": ")
        .append("Tidak ada pertukaran antara ke-")
        .append(j_2011).append(" dan ke-")
        .append(j_2011 + 1).append("\n");
}

stepLog_2011.append("Hasil: ").append(arrayToString(array_2011)).append("\n\n");
stepArea_2011.append(stepLog_2011.toString());

updateLabels();

j_2011++;

if (j_2011 >= array_2011.length - 1 - i_2011) {
    j_2011 = 0;
    i_2011++;
}

stepCount_2011++;

if (i_2011 >= array_2011.length - 1) {
    sorting_2011 = false;
    stepButton_2011.setEnabled(false);
    resetHighlights();
}

```

```

        for (JLabel label_2011 : labelArray_2011) {

            label_2011.setBackground(Color.GREEN);

        }

        JOptionPane.showMessageDialog(this, "Sorting selesail!");

    }

}

private void updateLabels() {

    for (int k_2011 = 0; k_2011 < array_2011.length; k_2011++) {

        labelArray_2011[k_2011].setText(String.valueOf(array_2011[k_2011]));

    }

}

private void resetHighlights() {

    for (JLabel label_2011 : labelArray_2011) {

        label_2011.setBackground(Color.WHITE);

    }

}

private void reset() {

    inputField_2011.setText("");

    panelArray_2011.removeAll();

    panelArray_2011.revalidate();

    panelArray_2011.repaint();

    stepArea_2011.setText("");

    stepButton_2011.setEnabled(false);

    sorting_2011 = false;

    i_2011 = 0;

    j_2011 = 0;

    stepCount_2011 = 1;

}

private String arrayToString(int[] arr) {

    StringBuilder sb_2011 = new StringBuilder();

    for (int k_2011 = 0; k_2011 < arr.length; k_2011++) {

        sb_2011.append(arr[k_2011]);

        if (k_2011 < arr.length - 1) {

            sb_2011.append(", ");

        }

    }

}

```



```
        return sb_2011.toString();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                new BubbleSortGUI_2511532011().setVisible(true);
            }
        });
    }
}
```

```

package pekan8_251132011;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;

import java.util.LinkedList;
import java.util.Queue;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

public class MergeSortGui_2511532011 extends JFrame {

    private static final long serialVersionUID_2011 = 1L;

    private int[] array_2011;
    private JLabel[] labelArray_2011;
    private JButton stepButton_2011, resetButton_2011, setButton_2011;
    private JTextField inputField_2011;
    private JPanel panelArray_2011;
    private JTextArea stepArea_2011;

    private int i_2011, j_2011, k_2011;
    private int left_2011, mid_2011, right_2011;
    private int[] temp_2011;

    private boolean isMerging_2011 = false;
    private boolean copying_2011 = false;
    private int stepCount_2011 = 1;

    private Queue<int[]> mergeQueue_2011 = new LinkedList<>();

    public MergeSortGui_2511532011() {
        setTitle("Merge Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel_2011 = new JPanel(new FlowLayout());
        inputField_2011 = new JTextField(30);
        setButton_2011 = new JButton("Set Array");
        inputPanel_2011.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
        inputPanel_2011.add(inputField_2011);
        inputPanel_2011.add(setButton_2011);

        // Panel array visual
        panelArray_2011 = new JPanel();
        panelArray_2011.setLayout(new FlowLayout());

        // Panel kontrol
        JPanel controlPanel_2011 = new JPanel();
        stepButton_2011 = new JButton("Langkah Selanjutnya");
        resetButton_2011 = new JButton("Reset");
        stepButton_2011.setEnabled(false);
        controlPanel_2011.add(stepButton_2011);
        controlPanel_2011.add(resetButton_2011);

        // Area teks untuk log langkah-langkah
        stepArea_2011 = new JTextArea(8, 60);
        stepArea_2011.setEditable(false);
        stepArea_2011.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scrollPane_2011 = new JScrollPane(stepArea_2011);

        // Tambahkan panel ke frame
        add(inputPanel_2011, BorderLayout.NORTH);
        add(panelArray_2011, BorderLayout.CENTER);
        add(controlPanel_2011, BorderLayout.SOUTH);
    }

```

```

add(scrollPane_2011, BorderLayout.EAST);

// Event Set Array
setButton_2011.addActionListener(e_2011 -> setArrayFromInput_2011());

// Event Langkah Selanjutnya
stepButton_2011.addActionListener(e_2011 -> performStep_2011());

// Event Reset
resetButton_2011.addActionListener(e_2011 -> reset_2011());
}

private void setArrayFromInput_2011() {
    String text_2011 = inputField_2011.getText().trim();
    if (text_2011.isEmpty()) return;

    String[] parts_2011 = text_2011.split(",");
    array_2011 = new int[parts_2011.length];

    try {
        for (int i_2011 = 0; i_2011 < parts_2011.length; i_2011++) {
            array_2011[i_2011] = Integer.parseInt(parts_2011[i_2011].trim());
        }
    } catch (NumberFormatException e_2011) {
        JOptionPane.showMessageDialog(this, "Masukkan hanya angka!",
            "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    labelArray_2011 = new JLabel[array_2011.length];
    panelArray_2011.removeAll();

    for (int i_2011 = 0; i_2011 < array_2011.length; i_2011++) {
        labelArray_2011[i_2011] = new JLabel(String.valueOf(array_2011[i_2011]));
        labelArray_2011[i_2011].setFont(new Font("Arial", Font.BOLD, 24));
        labelArray_2011[i_2011].setOpaque(true);
        labelArray_2011[i_2011].setBackground(Color.WHITE);
        labelArray_2011[i_2011].setBorder(BorderFactory.createLineBorder(Color.BLACK));
        labelArray_2011[i_2011].setPreferredSize(new Dimension(50, 50));
        labelArray_2011[i_2011].setHorizontalAlignment(SwingConstants.CENTER);
        panelArray_2011.add(labelArray_2011[i_2011]);
    }

    mergeQueue_2011.clear();
    generateMergeSteps_2011(0, array_2011.length - 1);

    stepButton_2011.setEnabled(true);
    stepArea_2011.setText("");
    stepCount_2011 = 1;
    isMerging_2011 = false;
    copying_2011 = false;

    panelArray_2011.revalidate();
    panelArray_2011.repaint();
}

private void generateMergeSteps_2011(int left_2011, int right_2011) {
    if (left_2011 < right_2011) {
        int mid_2011 = left_2011 + (right_2011 - left_2011) / 2;

        generateMergeSteps_2011(left_2011, mid_2011);
        generateMergeSteps_2011(mid_2011 + 1, right_2011);

        mergeQueue_2011.add(new int[] { left_2011, mid_2011, right_2011 });
    }
}

private void performStep_2011() {
    resetHighlights_2011();

    if (!isMerging_2011 && !mergeQueue_2011.isEmpty()) {
        int[] range_2011 = mergeQueue_2011.poll();

        left_2011 = range_2011[0];
        mid_2011 = range_2011[1];
        right_2011 = range_2011[2];

        temp_2011 = new int[right_2011 - left_2011 + 1];

        i_2011 = left_2011;
        j_2011 = mid_2011 + 1;
    }
}

```

```

        k_2011 = 0;

        copying_2011 = false;
        isMerging_2011 = true;

        stepArea_2011.append(
            "Langkah " + stepCount_2011++
            + ": Mulai merge dari "
            + left_2011 + " ke "
            + right_2011 + "\n"
        );

        return;
    }

    if (isMerging_2011 && !copying_2011) {
        if (i_2011 <= mid_2011 && j_2011 <= right_2011) {
            labelArray_2011[i_2011].setBackground(Color.CYAN);
            labelArray_2011[j_2011].setBackground(Color.CYAN);

            if (array_2011[i_2011] <= array_2011[j_2011]) {
                temp_2011[k_2011++] = array_2011[i_2011++];
            } else {
                temp_2011[k_2011++] = array_2011[j_2011++];
            }

            stepArea_2011.append(
                "Langkah " + stepCount_2011++
                + ": Bandingkan dan salin elemen\n"
            );

            return;
        } else if (i_2011 <= mid_2011) {
            temp_2011[k_2011++] = array_2011[i_2011++];

            stepArea_2011.append(
                "Langkah " + stepCount_2011++
                + ": Salin sisa kiri\n"
            );

            return;
        } else if (j_2011 <= right_2011) {
            temp_2011[k_2011++] = array_2011[j_2011++];

            stepArea_2011.append(
                "Langkah " + stepCount_2011++
                + ": Salin sisa kanan\n"
            );

            return;
        } else {
            copying_2011 = true;
            k_2011 = 0;
            return;
        }
    }

    if (copying_2011 && k_2011 < temp_2011.length) {
        array_2011[left_2011 + k_2011] = temp_2011[k_2011];
        labelArray_2011[left_2011 + k_2011].setText(String.valueOf(temp_2011[k_2011]));
        labelArray_2011[left_2011 + k_2011].setBackground(Color.GREEN);
        k_2011++;

        stepArea_2011.append("Langkah " + stepCount_2011++ + ": Tempelkan ke array utama\n");
        return;
    }

    if (copying_2011 && k_2011 == temp_2011.length) {
        isMerging_2011 = false;
        copying_2011 = false;
    }

    if (mergeQueue_2011.isEmpty() && !isMerging_2011) {
        stepArea_2011.append("Selesai.\n");
        stepButton_2011.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Merge Sort selesai!");
    }
}

```

```

private void resetHighlights_2011() {
    if (labelArray_2011 == null) return;

    for (JLabel label_2011 : labelArray_2011) {
        label_2011.setBackground(Color.WHITE);
    }
}

private void reset_2011() {
    inputField_2011.setText("");
    panelArray_2011.removeAll();
    panelArray_2011.revalidate();
    panelArray_2011.repaint();
    stepArea_2011.setText("");
    stepButton_2011.setEnabled(false);
    mergeQueue_2011.clear();
    isMerging_2011 = false;
    copying_2011 = false;
    stepCount_2011 = 1;
}

public static void main(String[] args_2011) {
    SwingUtilities.invokeLater(() -> {
        MergeSortGui_2511532011 frame_2011 = new MergeSortGui_2511532011();
        frame_2011.setVisible(true);
    });
}
}

```

## 2.2 Tugas Pratikum Lagu

Saya memilih algoritma quicksort karena implementasinya sudah pernah dijelaskan saat praktikum dan performanya lebih baik

```
package pekan8_251132011;

public class Lagu_2511532011 {
    private String judul_2011;
    private String penyanyi_2011;
    private int durasi_2011;

    public Lagu_2511532011(String j_2011, String p_2011,
int d_2011 ) {
        this.judul_2011 = j_2011;
        this.penyanyi_2011 = p_2011;
        this.durasi_2011 = d_2011;
    }

    public String getJ_2011() {return judul_2011;}
    public String getP_2011() {return penyanyi_2011;}
    public int getD_2011() {return durasi_2011;}

    public void setJ_2011(String j_2011) {
        this.judul_2011 = j_2011;
    }

    public void setP_2011(String p_2011) {
        this.penyanyi_2011 = p_2011;
    }

    public void setD_2011(int d_2011) {
        this.durasi_2011 = d_2011;
    }

    public String toString_2011() {
        return judul_2011 + " - " + penyanyi_2011 + " ("
+ durasi_2011 + " detik)";
    }
}
```

Pembahasan Class Lagu:

Deklarasi Kelas

`public class Lagu_2511532011`: Mendefinisikan kelas publik yang berfungsi sebagai cetak biru (blueprint) untuk objek data lagu.

Atribut Privat (Enkapsulasi)

`private String judul_2011`:: Variabel instance untuk menyimpan judul lagu. Modifier `private` membatasi akses langsung dari luar kelas.

`private String penyanyi_2011`:: Variabel instance untuk menyimpan nama penyanyi/artis.

`private int durasi_2011`:: Variabel instance bertipe integer untuk menyimpan durasi lagu dalam satuan detik.

Konstruktur (Inisialisasi Objek)

`public Lagu_2511532011(String j_2011, String p_2011, int d_2011)`: Konstruktur publik yang menerima tiga parameter untuk inisialisasi objek lagu baru.

`this.judul_2011 = j_2011`:: Assign nilai parameter judul ke atribut instance menggunakan keyword `this`.

`this.penyanyi_2011 = p_2011`:: Assign nilai parameter penyanyi ke atribut instance.

`this.durasi_2011 = d_2011`:: Assign nilai parameter durasi ke atribut instance.

Metode Getter (Akses Data)

`public String getJ_2011()` { return `judul_2011`; }: Mengembalikan nilai judul lagu. Memungkinkan kelas lain membaca atribut privat secara terkontrol.

`public String getP_2011()` { return `penyanyi_2011`; }: Mengembalikan nilai nama penyanyi.

`public int getD_2011()` { return `durasi_2011`; }: Mengembalikan nilai durasi lagu dalam detik.

Metode Setter (Modifikasi Data)

`public void setJ_2011(String j_2011)`: Mengubah nilai judul lagu setelah objek dibuat.

`public void setP_2011(String p_2011)`: Mengubah nilai nama penyanyi.

`public void setD_2011(int d_2011)`: Mengubah nilai durasi lagu.

```

package pekan8_251132011;

public class Sorting_2511532011 {
    Lagu_2511532011[] listLagu_2011 = new
    Lagu_2511532011[20];
    int jumlah_2011 = 0;

    public void inputData_2011() {
        listLagu_2011[0] = new Lagu_2511532011("Neon
Rain", "Neoni", 165);
        listLagu_2011[1] = new Lagu_2511532011("Take
Off", "Quavo & Offset", 195);
        listLagu_2011[2] = new Lagu_2511532011("Fake It",
"Seether", 235);
        listLagu_2011[3] = new Lagu_2511532011("Anti
Hero", "Taylor Swift", 200);
        listLagu_2011[4] = new Lagu_2511532011("Back To
Be Friend", "Noah Cyrus", 210);
        listLagu_2011[5] = new Lagu_2511532011("Stay With
Me", "Sam Smith", 172);
        listLagu_2011[6] = new Lagu_2511532011("Clarity",
"Zedd ft. Foxes", 271);
        jumlah_2011 = 7;
    }

    public void tampilData_2011() {
        for (int i_2011 = 0; i_2011 < jumlah_2011;
++i_2011) {
            System.out.println((i_2011 + 1) + ". " +
listLagu_2011[i_2011].toString_2011());
        }
    }

    static void swap_2011(Lagu_2511532011[] arr_2011, int
i_2011, int j_2011) {
        Lagu_2511532011 temp_2011 = arr_2011[i_2011];
        arr_2011[i_2011] = arr_2011[j_2011];
        arr_2011[j_2011] = temp_2011;
    }

    static void medianOfThree_2011(Lagu_2511532011[]
arr_2011, int low_2011, int high_2011) {
        int mid_2011 = low_2011 + (high_2011 - low_2011)
/ 2;

```



```

        if (arr_2011[low_2011].getD_2011() >
arr_2011[mid_2011].getD_2011()) {
            swap_2011(arr_2011, low_2011, mid_2011);
        }
        if (arr_2011[low_2011].getD_2011() >
arr_2011[high_2011].getD_2011()) {
            swap_2011(arr_2011, low_2011, high_2011);
        }
        if (arr_2011[mid_2011].getD_2011() >
arr_2011[high_2011].getD_2011()) {
            swap_2011(arr_2011, mid_2011, high_2011);
        }
        swap_2011(arr_2011, mid_2011, high_2011);
    }

    static int partition_2011(Lagu_2511532011[] arr_2011,
int low_2011, int high_2011) {
        medianOfThree_2011(arr_2011, low_2011,
high_2011);

        int pivot_2011 = arr_2011[high_2011].getD_2011();
        int i_2011 = (low_2011 - 1);

        for (int j_2011 = low_2011; j_2011 <= high_2011 -
1; j_2011++) {
            if (arr_2011[j_2011].getD_2011() <
pivot_2011) {
                i_2011++;
                swap_2011(arr_2011, i_2011, j_2011);
            }
        }
        swap_2011(arr_2011, i_2011 + 1, high_2011);
        return (i_2011 + 1);
    }

    static void quickSort_2011(Lagu_2511532011[] arr_2011,
int low_2011, int high_2011) {
        if (low_2011 < high_2011) {
            int pi_2011 = partition_2011(arr_2011,
low_2011, high_2011);
            quickSort_2011(arr_2011, low_2011, pi_2011
- 1);
            quickSort_2011(arr_2011, pi_2011 + 1,
high_2011);
        }
    }

```

```

        public void quickSort_2011() {
            if (jumlah_2011 > 1) {
                quickSort_2011(listLagu_2011, 0,
jumlah_2011 - 1);
            }
        }

        public static void main(String[] args) {
            Sorting_2511532011 playlist_2011 = new
Sorting_2511532011();

            System.out.println("=== Sorting Playlist NIM:
2511532011 ===");
            System.out.println("Pilih Algoritma (1=Shell,
2=Quick, 3=Merge): ");
            playlist_2011.inputData_2011();

            System.out.println("\nData Sebelum Sorting:");
            playlist_2011.tampilData_2011();

            playlist_2011.quickSort_2011();

            System.out.println("\nData Setelah Quick Sort
(Durasi Asc):");
            playlist_2011.tampilData_2011();
        }
    }
}

```

Pembahasan Class Sorting:

Atribut Instance (Manajemen Data)

Lagu\_2511532011[] listLagu\_2011 = new Lagu\_2511532011[20];: Array statis berkapasitas 20 elemen untuk menyimpan referensi objek lagu. Menggunakan array primitif alih-alih ArrayList menunjukkan pendekatan memori terkontrol.

int jumlah\_2011 = 0;: Variabel penanda ukuran logis array. Mencegah iterasi pada elemen null di index yang belum terisi.

Metode inputData\_2011() (Inisialisasi Data)

Mengisi array index 0 sampai 6 dengan instance objek Lagu\_2511532011 yang telah didefinisikan hardcoded.

jumlah\_2011 = 7;: Menetapkan batas valid data. Hanya 7 slot pertama yang akan diproses oleh metode sorting dan display.

Metode tampilData\_2011() (Display)

for (int i\_2011 = 0; i\_2011 < jumlah\_2011; ++i\_2011): Loop iterasi terbatas pada ukuran logis, bukan kapasitas array (20).

System.out.println((i\_2011 + 1) + ". " + listLagu\_2011[i\_2011].toString\_2011());:

Mencetak daftar bernomor dengan memanggil metode representasi string kustom pada objek lagu.

Metode swap\_2011() (Helper Pertukaran Objek)

static void swap\_2011(Lagu\_2511532011[] arr\_2011, int i\_2011, int j\_2011):

Metode statis untuk menukar posisi dua referensi objek dalam array.

Menggunakan variabel temporary temp\_2011 bertipe Lagu\_2511532011. Karena Java melewati referensi secara by-reference, penukaran ini langsung memodifikasi array asli.

Metode medianOfThree\_2011() (Optimasi Pivot)

static void medianOfThree\_2011(...): Memilih pivot yang lebih representatif untuk mencegah worst-case pada data durasi yang sudah terurut.

if (arr\_2011[low\_2011].getD\_2011() > arr\_2011[mid\_2011].getD\_2011()):

Perbandingan dilakukan berdasarkan durasi menggunakan getter .getD\_2011(), bukan nilai primitif langsung.

Tiga perbandingan mengurutkan relatif elemen low, mid, high.

swap\_2011(arr\_2011, mid\_2011, high\_2011);: Memindahkan elemen median (nilai tengah) ke posisi high agar kompatibel dengan skema partisi Lomuto.

Metode partition\_2011() (Pembagian Partisi)

static int partition\_2011(...): Inti algoritma Quick Sort yang membagi array menjadi dua bagian berdasarkan pivot.

int pivot\_2011 = arr\_2011[high\_2011].getD\_2011();: Nilai pivot diambil dari durasi lagu di index high.

int i\_2011 = (low\_2011 - 1);: Penanda batas kanan dari sub-array yang berisi lagu dengan durasi < pivot.

if (arr\_2011[j\_2011].getD\_2011() < pivot\_2011): Jika durasi lagu saat ini lebih kecil dari pivot, increment i\_2011 dan tukar dengan posisi j\_2011.

swap\_2011(arr\_2011, i\_2011 + 1, high\_2011);: Menempatkan pivot di posisi final yang benar (i+1).

return (i\_2011 + 1);: Mengembalikan index pivot untuk digunakan dalam rekursi.

Metode quickSort\_2011() (Rekursi Statis)

static void quickSort\_2011(...): Implementasi Divide-and-Conquer murni.

if (low\_2011 < high\_2011): Base case. Berhenti jika sub-array berukuran 0 atau 1.

int pi\_2011 = partition\_2011(...): Dapatkan posisi final pivot.

quickSort\_2011(arr\_2011, low\_2011, pi\_2011 - 1);: Urutkan sisi kiri (durasi lebih kecil).

quickSort\_2011(arr\_2011, pi\_2011 + 1, high\_2011);: Urutkan sisi kanan (durasi lebih besar).

Metode quickSort\_2011() (Wrapper Instance)

public void quickSort\_2011(): Metode publik yang menyediakan antarmuka bersih untuk kelas utama.

if (jumlah\_2011 > 1): Guard clause. Tidak perlu mengurutkan jika data kosong atau hanya 1 lagu.

Memanggil versi statis dengan batas 0 hingga jumlah\_2011 - 1, sehingga hanya data valid yang diproses.

Metode main() (Alur Eksekusi)

Sorting\_2511532011 playlist\_2011 = new Sorting\_2511532011();: Instansiasi objek pengelola.

playlist\_2011.inputData\_2011();: Muat 7 data lagu ke array.

playlist\_2011.quickSort\_2011();: Jalankan pengurutan berdasarkan durasi secara ascending.

Mencetak kondisi sebelum dan sesudah untuk verifikasi hasil.



## **BAB III**

### **KESIMPULAN**

#### **3.1 Kesimpulan**

Berdasarkan hasil praktikum dan pembahasan, telah berhasil diimplementasikan tiga algoritma pengurutan dasar yaitu Insertion Sort, Selection Sort, dan Bubble Sort menggunakan Java secara in-place serta dibangun aplikasi GUI berbasis Java Swing untuk memvisualisasikan proses Insertion Sort secara interaktif langkah demi langkah. Praktikum ini menegaskan pemahaman mendalam mengenai kompleksitas waktu  $O(n^2)$ , logika perulangan bersarang, dan manipulasi array melalui variabel temporary, sekaligus menunjukkan kemampuan integrasi komponen GUI seperti JTextField, JLabel, dan JTextArea untuk menghubungkan logika algoritma backend dengan antarmuka pengguna, sehingga tujuan praktikum dalam memberikan pemahaman komprehensif mengenai algoritma pengurutan serta keterampilan teknis pembangunan aplikasi visualisasi tercapai secara optimal.

## **DAFTAR PUSTAKA**

[1]W3schools,“JavaDataStructur”2026[Daring]. Tersediapada:[https://www.w3schools.com/java/java\\_data\\_structures.asp](https://www.w3schools.com/java/java_data_structures.asp). [Diakses:20-Meil-2026]

